

IQC 2025 | 16/4/2025 Roberto Bruni

TOWARDS A QUANTUM PROGRAMMING LANGUAGE

Peter Selinger Sect. 1-5

Quantum computation? gates + circuit
data flows, but control flow?
parametrized families of circuits

Selinger moves the perspective to programming languages
data flow + control flow how realization independent

Slogan "Quantum data, Classical Control"

Quantum superposition can be exploited

no quantum branching
no statement superposition

functional approach: statements transform input to output
atomic / composite

graphical language: Flow charts

loops + recursion

statically typed: program well-formedness at "compilation"

no runtime check
no runtime errors

intuitive denotational semantics (pictorial, visual format)

fundamental operations: unitary transformations
measurement

Kuill's QRAM model

classical device sends sequence of quantum operations
to a quantum device

↓
quantum circuit

the quantum device sends back the result of some
measurement

the classical controller then decides what to do next

Quantum algorithms: interleaving quantum and classical
operations

Assume perfect HW is available no noise
no error ...

Approach: states are not state vectors

but density matrices (due to
Von Neumann)

several advantages: pure states $|\psi\rangle$

as well as mixed states $\underline{\alpha_1}|\psi_1\rangle + \dots + \underline{\alpha_m}|\psi_m\rangle$

Density matrices form a complete partial order
as needed to solve recursive equations by

least fixpoint computation

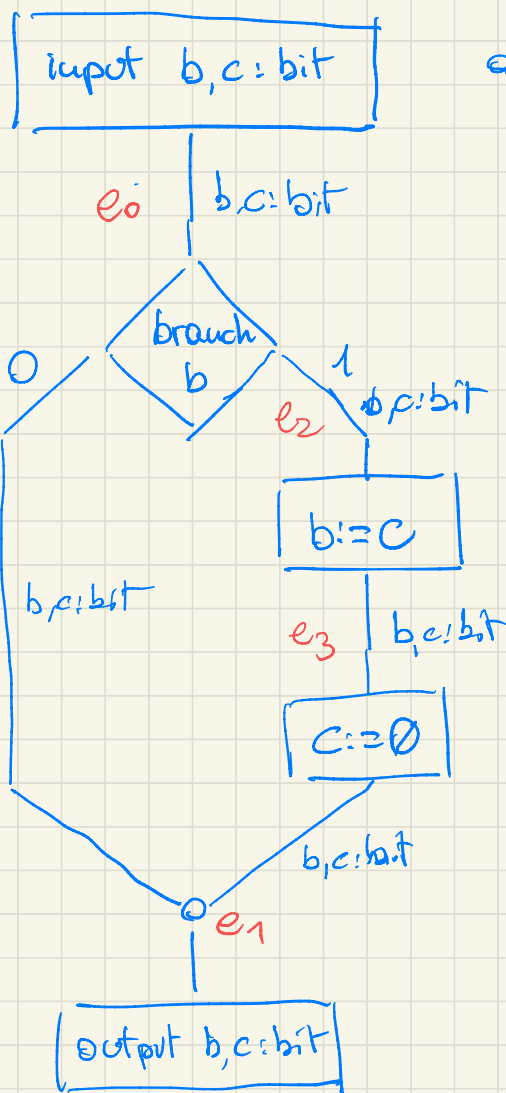
$$\underset{\uparrow}{x} = \underset{=}{f}(\underset{=}{x})$$

abstract from global phase

subprobability distribution to handle divergence

reduce the number of square roots symbols!

Classical Flowcharts



meaning:
a map from input to output

u

input b c	output b c
0 0	0 0
0 1	0 1
1 0	0 0
1 1	1 0

State of the program

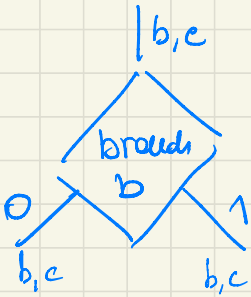
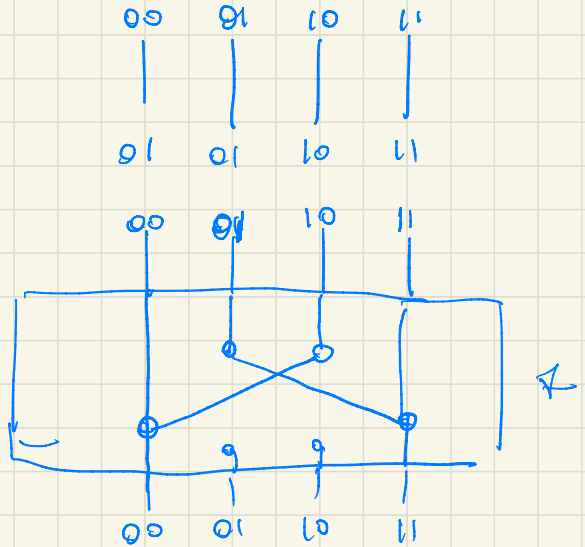
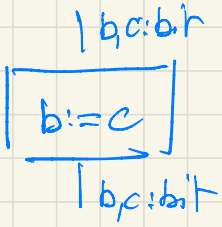
$\langle \text{edge}, p \rangle$

in the flowchart $p: \text{Var} \rightarrow \text{Values}$

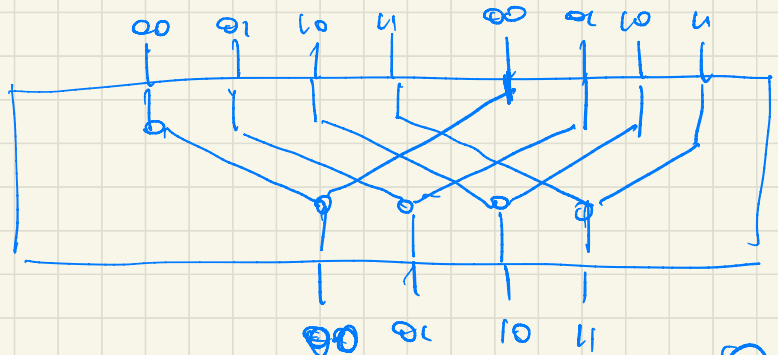
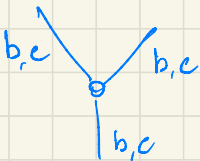
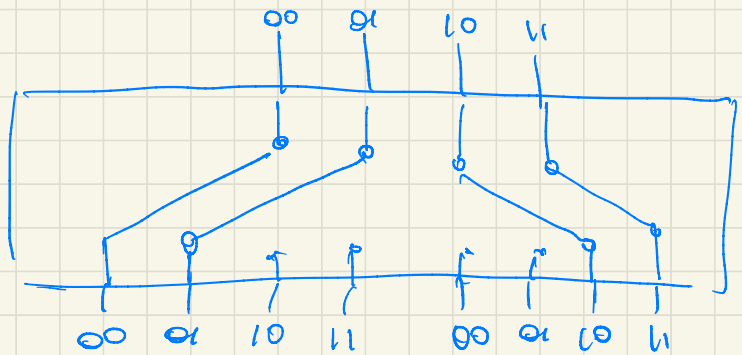
$\langle e_0, \boxed{bc \mapsto 10} \rangle \rightarrow \langle e_2, bc \mapsto 10 \rangle \rightarrow \langle e_3, bc \mapsto 00 \rangle$
 $\rightarrow \langle e_1, \boxed{bc \mapsto 00} \rangle$

edges vs states

↓ $b, c: \text{bit}$



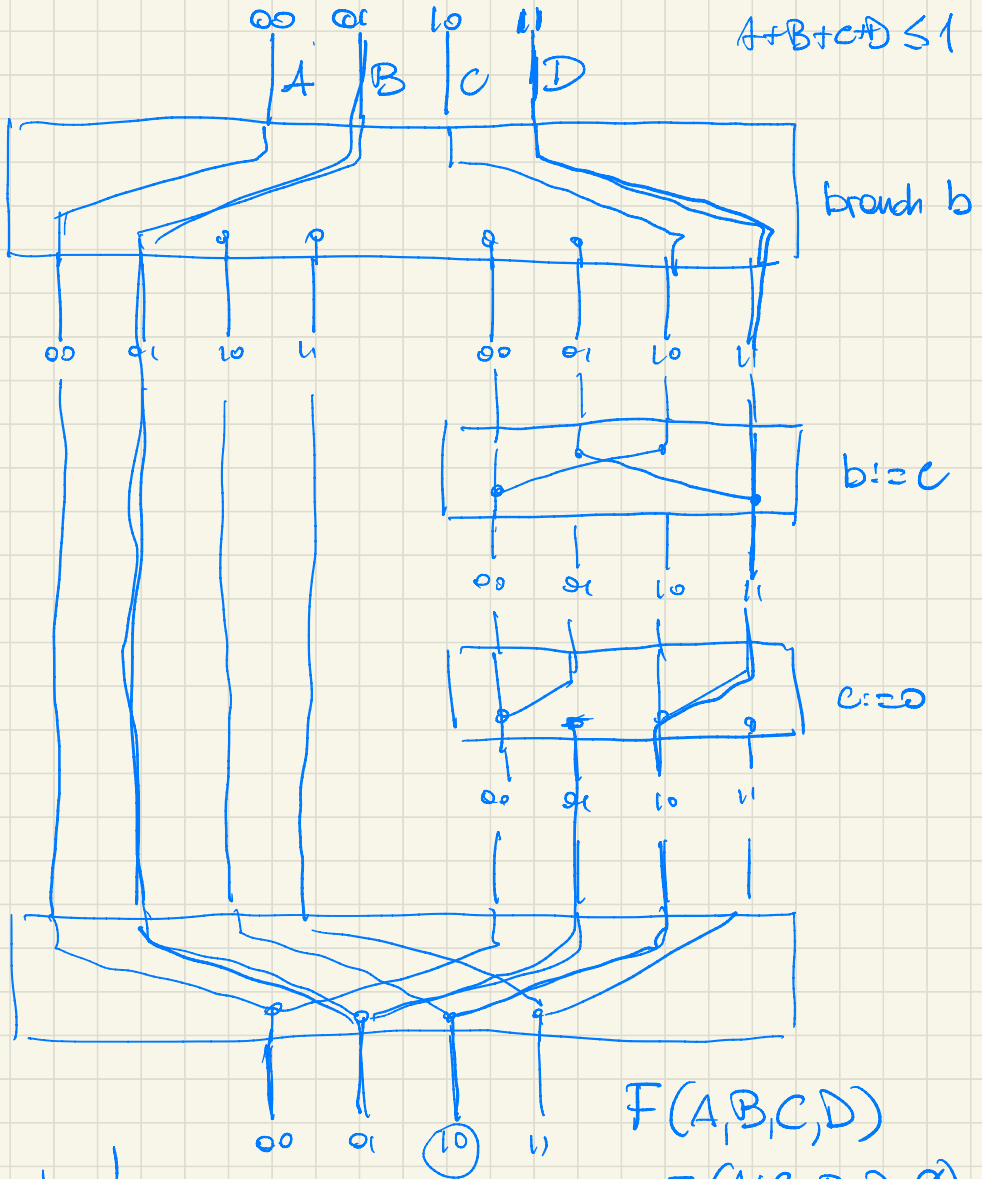
→



Probabilistic Flowcharts

$$0 \leq A, B, C, D \leq 1$$

$$A+B+C+D \leq 1$$



* Likelihood transformation!

input

$$p(00) = A$$

$$p(01) = B$$

$$p(10) = C$$

$$p(11) = D$$

output

$$p(00) = A+C$$

$$p(01) = B$$

$$p(10) = D$$

$$p(11) = \emptyset$$

$$F(A, B, C, D)$$

$$= (A+C, B, D, \emptyset)$$

$$F(1, 0, 0, 0)$$

$$F(0, 1, 0, 0)$$

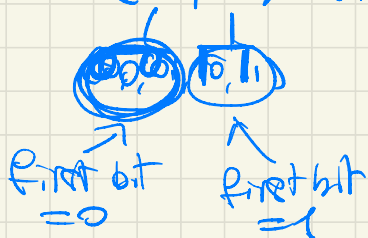
$$F(0, 0, 1, 0)$$

$$F(0, 0, 0, 1)$$

Γ a typing context b, c : bit edge labels syntax

A, B, C, D (tuples of probabilities) annotations semantics

we write (A, B) when A, B are tuple of same length



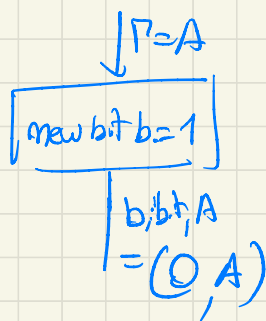
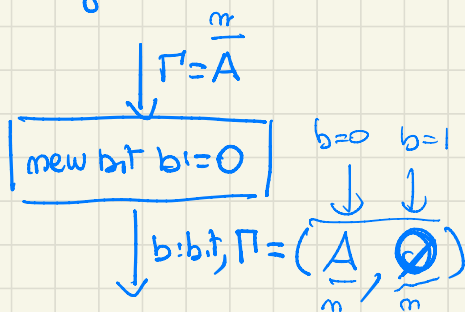
$\Gamma = A$ label + annotation

Syntax + Semantics

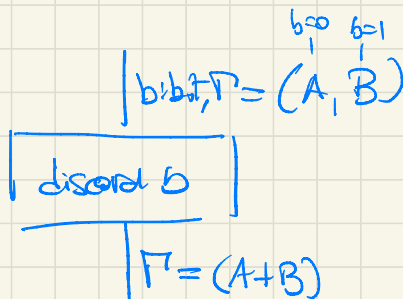
well-formed

meaning

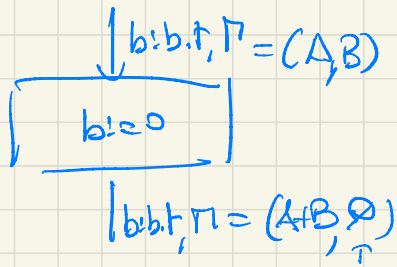
Allocate bit



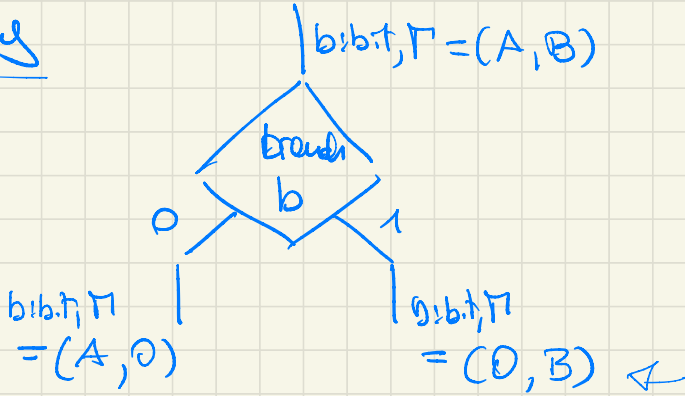
Discard bit



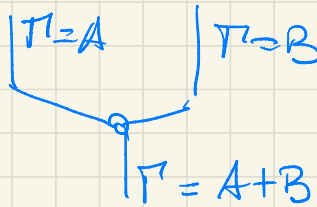
Assignment



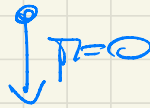
Branching



Merge



initial



Permutation

$$b_1:bit, \dots, b_m:bit = (\overset{0 \dots 0}{A_0} \dots \overset{1 \dots 1}{A_{2^m-1}})$$

permutation ϕ

$$\phi: \{1, \dots, m\} \rightarrow \{1, \dots, m\}$$

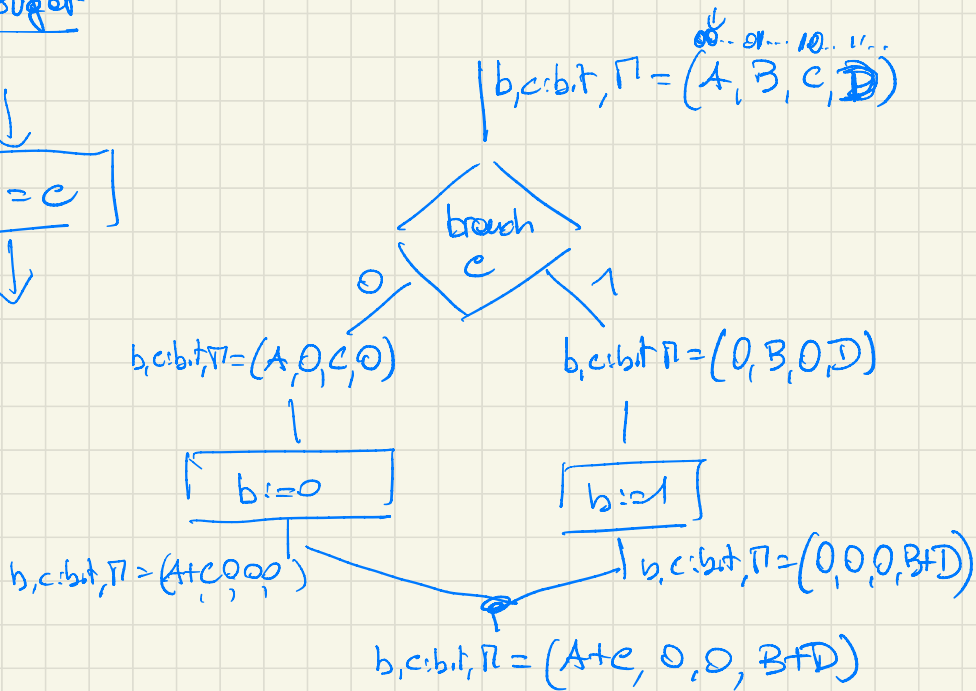
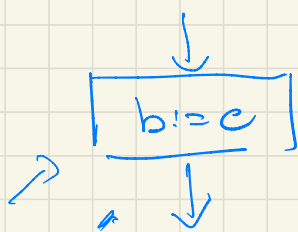
$$b_{\phi(1)} \dots b_{\phi(m)}:bit$$

$$= (A_{2^{\phi(1)}} \dots A_{2^{\phi(2^m-1)}})$$

$$2^\phi: \{0, 1\}^m \rightarrow \{0, 1\}^m$$

$$2^\phi(x_1, \dots, x_m) = (x_{\phi(1)}, \dots, x_{\phi(m)})$$

Syntax Sugar



Quantum Flow Charts

$$x \in \{0,1\}$$
$$q = S q$$

+ a new type qbit — } $q: \text{qbit}$

+ Unitary transformation —

+ Measurements

} branching statements
driven by measurement

